

Implementação de uma Rede Neural em um Sistema de Controle de *Ball and Beam*

Rota do Visionário

1 Introdução

A robótica tem sido desenvolvida desde o século XX, com a criação de sistemas constituídos por sensores e atuadores capazes de executar tarefas que vão desde operações simples, como mover um objeto de um local a outro, até aplicações altamente complexas, incluindo sistemas autônomos baseados em dados.

Nesse contexto, surge uma necessidade intrínseca à própria proposta de robôs inteligentes: autonomia. Como desenvolver um sistema capaz de adaptar sua resposta conforme as informações coletadas do ambiente, seguindo uma tarefa pré-determinada? É nesse ponto que entram os Sistemas de Controle.

Os métodos clássicos de controle (como o controle por alocação de polos no espaço de estados) oferecem uma abordagem analítica e interpretável para esse problema (Ogata; Souza; Tannuri, 2011). A partir de um modelo matemático do sistema, é possível projetar um controlador que garante estabilidade e desempenho dentro de condições bem definidas. No entanto, essa abordagem depende da qualidade do modelo e pode ser limitada diante de não-linearidades e incertezas do sistema real.

Paralelamente, o campo do Aprendizado de Máquina trouxe uma perspectiva complementar: em vez de modelar explicitamente determinado problema, aprender seu comportamento a partir de dados. Dentro desse campo, as Redes Neurais Artificiais se destacam pela capacidade de aproximar funções complexas com alta precisão, ao custo de maior demanda computacional e menor interpretabilidade.

Uma abordagem recente busca unir o melhor dos dois mundos: as *Physics-Informed Neural Networks* (PINNs), redes neurais que incorporam conhecimento físico do sistema diretamente em sua arquitetura ou função de perda (Raissi; Perdikaris; Karniadakis, 2019). Essa hibridização permite que a rede generalize melhor com menos dados, ao mesmo tempo que respeita as leis que governam o sistema.

O presente trabalho utiliza o sistema Ball and Beam como plataforma experimental, (um problema clássico de controle, linearizado e instável em malha aberta), para comparar o desempenho de um controlador projetado por alocação de polos com o de uma rede neural de arquitetura MLP inspirada em PINN. O objetivo é avaliar se a abordagem baseada em uma arquitetura de rede neural consegue reproduzir ou superar o desempenho do método clássico em termos de tempo de acomodação, overshoot e erro em regime permanente.

2 Metodologia

2.1 Materiais

Para a construção da estrutura física do *Ball and Beam*, utilizou-se um modelo de impressão em 3D, disponibilizada no site GrabCAD (Ochoa, 2022). A impressão foi realizada na impressora Prusa i3 MK3, com filamento PLA na cor cinza grafite. O controlador foi embarcado em uma placa ESP32-S3, devido à sua maior capacidade de armazenamento. As medições foram obtidas por meio de um sensor de distância VL53L0X, e o atuador empregado para controlar a inclinação da viga foi o servomotor MG996R. O circuito foi montado em uma placa de prototipação (*protoboard*), com conexões realizadas por *jumper*s. O microcontrolador foi alimentado via cabo USB conectado ao computador, enquanto o servomotor foi alimentado por uma fonte externa, uma vez que demanda corrente superior à que a ESP32 é capaz de fornecer. A bola utilizada foi uma bola de tênis de mesa. Todos os materiais utilizados estão ilustrados na Figura 1.

2.2 Métodos

Montagem do *Ball and Beam*

A primeira etapa consistiu na montagem da planta física seguindo o modelo 3D, com a única adaptação no encaixe do sensor: substituiu-se o sensor ultrassônico, previsto no modelo original, por um sensor a laser.

Em seguida, foram realizados testes com o controlador clássico, utilizando-se a extensão PlatformIO no Visual Studio Code, que possibilitou a integração de código em C++ - destinado à comunicação com o microcontrolador - e em Python - destinado à integração com a rede neural.

Por fim, procedeu-se ao treinamento da rede neural.

Formulação da rede neural

A proposta inicial deste trabalho era empregar uma *Physics-Informed Neural Network* (Raissi; Perdikaris; Karniadakis, 2019), conforme a formulação de Raissi et al. (2017), na qual uma rede aproxima diretamente a solução de uma equação diferencial e a função de perda inclui o resíduo dessa equação avaliado em pontos de colocação. Contudo, ao longo da implementação, identificou-se uma limitação fundamental: o sinal disponível no sistema real é a posição $x(t)$, e não a aceleração $\ddot{x}$. A derivação numérica do sinal de posição — já ruidoso — produziu estimativas de aceleração próximas de zero, inviabilizando seu uso como dado de supervisão para o resíduo físico. Diante disso, adotou-se uma abordagem alternativa, descrita a seguir.

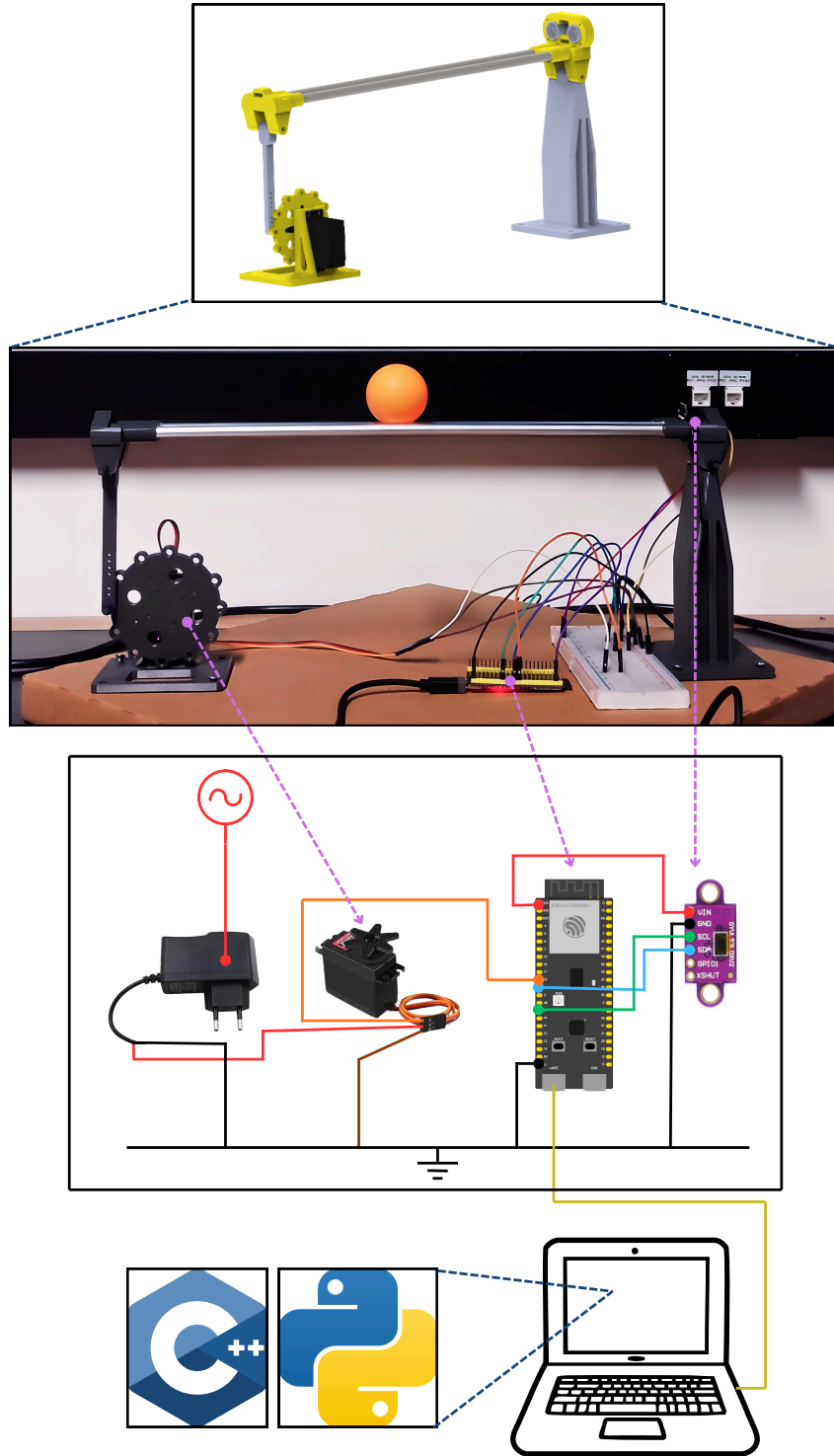


Figure 1: Fluxo do processo de implementação da rede neural na planta do *Ball and Beam*.

Arquitetura

Utilizou-se uma rede *Multilayer Perceptron* (MLP) com entradas $[x, \dot{x}, x_{\text{ref}}, e_{\text{int}}]$, onde e_{int} é o erro acumulado $\int (x_{\text{ref}} - x) dt$, discretizado como $e_{\text{int},k+1} = e_{\text{int},k} + (x_{\text{ref}} - x_k) \Delta t$. A saída da rede é o ângulo de comando θ (em graus), limitado ao intervalo $[-60, 60]$.

Treinamento por simulação diferenciável

O treinamento baseia-se em **simulação diferenciável**: a rede é integrada a um simulador que propaga o estado do sistema pela equação linearizada do movimento,

$$\ddot{x} = \frac{5}{9} g \theta, \quad (1)$$

discretizada por Euler explícito com período $\Delta t = 0,05$ s. A cada iteração do otimizador, um lote de N condições iniciais $(x_0, \dot{x}_0, x_{\text{ref}})$ é amostrado aleatoriamente no espaço de operação; a rede produz a sequência de ações $\{\theta_k\}$ e o simulador propaga a trajetória $\{x_k\}$ por n passos. O gradiente da loss em relação aos parâmetros da rede é calculado por *backpropagation* através do grafo computacional do simulador, sem necessidade de dados experimentais coletados a priori.

Esta abordagem difere da PINN clássica em um aspecto central: a rede não aproxima a solução da EDO, mas sim uma *política de controle*; a física não aparece como resíduo penalizado em pontos de colocação, mas está embutida na dinâmica do simulador que gera as trajetórias de treinamento.

Função de perda

A função de perda combina dois termos principais:

$$\mathcal{L} = w_{\text{state}} \mathcal{L}_{\text{state}} + w_{\text{effort}} \mathcal{L}_{\text{effort}}, \quad (2)$$

onde:

$$\mathcal{L}_{\text{state}} = \frac{1}{Nn} \sum_{i,k} (x_{i,k} - x_{\text{ref},i})^2, \quad (3)$$

$$\mathcal{L}_{\text{effort}} = \frac{1}{Nn} \sum_{i,k} \theta_{i,k}^2. \quad (4)$$

O termo $\mathcal{L}_{\text{state}}$ penaliza o erro de rastreamento ao longo de toda a trajetória. O termo $\mathcal{L}_{\text{effort}}$ evita ações excessivas, análogo à penalização de controle em reguladores quadráticos lineares (LQR). Um terceiro termo, $\mathcal{L}_{\text{edo}}$, que verifica a consistência interna da trajetória com a equação (1), foi considerado durante o desenvolvimento mas descartado: como as trajetórias são geradas pelo próprio simulador baseado nessa equação, o resíduo é estruturalmente nulo durante o treinamento, tornando o termo redundante.

Otimização dos hiperparâmetros

O ajuste dos parâmetros de controladores clássicos é tipicamente feito de forma empírica — testando valores e observando o comportamento da planta. Uma das motivações da abordagem

com redes neurais é justamente a possibilidade, amplamente utilizada na literatura, de otimizar os hiperparâmetros por meio de algoritmos específicos. Neste trabalho, utilizou-se o framework Optuna para essa finalidade. Os intervalos de busca estão listados na Tabela 1.

Table 1: Intervalos de busca dos hiperparâmetros.

Hiperparâmetro	Mínimo	Máximo
L (camadas)	1	3
H (neurônios)	64	256
$batch\ size$	100	1000
η (learning rate)	10^{-4}	10^{-2}
ω_{state}	5	10
ω_{effort}	0,1	0,5
Ativação	Tanh, SiLU	

Os melhores parâmetros encontrados estão listados na Tabela 2.

Table 2: Melhores hiperparâmetros encontrados pelo Optuna.

Hiperparâmetro	Valor
L (camadas)	3
H (neurônios)	256
$batch\ size$	316
η (learning rate)	10^{-3}
ω_{state}	5,11
ω_{effort}	0,35
Ativação	SiLU

Exportação do modelo

Com os parâmetros otimizados, realizou-se um treinamento final a fim de salvar o modelo com todos os pesos calculados. Para a implantação na ESP32-S3, implementou-se um script em Python que converte os pesos e ativações da rede para um arquivo de cabeçalho em linguagem C (.h), com a inferência implementada manualmente — sem o uso de bibliotecas externas como TFLite. Essa escolha foi necessária dado que as operações de multiplicação matricial são executadas em tempo real pelo microcontrolador. Por isso, foi importante restringir os intervalos de otimização de forma a evitar redes excessivamente profundas ou largas, cujo custo computacional poderia inviabilizar a execução embarcada.

Com o modelo embarcado, realizou-se um ensaio comparativo entre o controlador clássico e o controlador por rede neural. O ensaio consistiu em alterar o *setpoint* algumas vezes e aguardar a estabilização da bola em regime permanente. Os dados foram coletados e, a partir deles, gerou-se dois painéis de análise.

3 Resultados

Os painéis de resultados dos ensaios estão apresentados nas Figuras 2 e 3. Qualitativamente, para posições próximas à extremidade móvel da viga — ou seja, mais distantes do sensor — o controle se mostrou ruidoso e instável em ambas as abordagens. Para posições abaixo de 15 cm, o controle por rede neural apresentou comportamento mais estável.

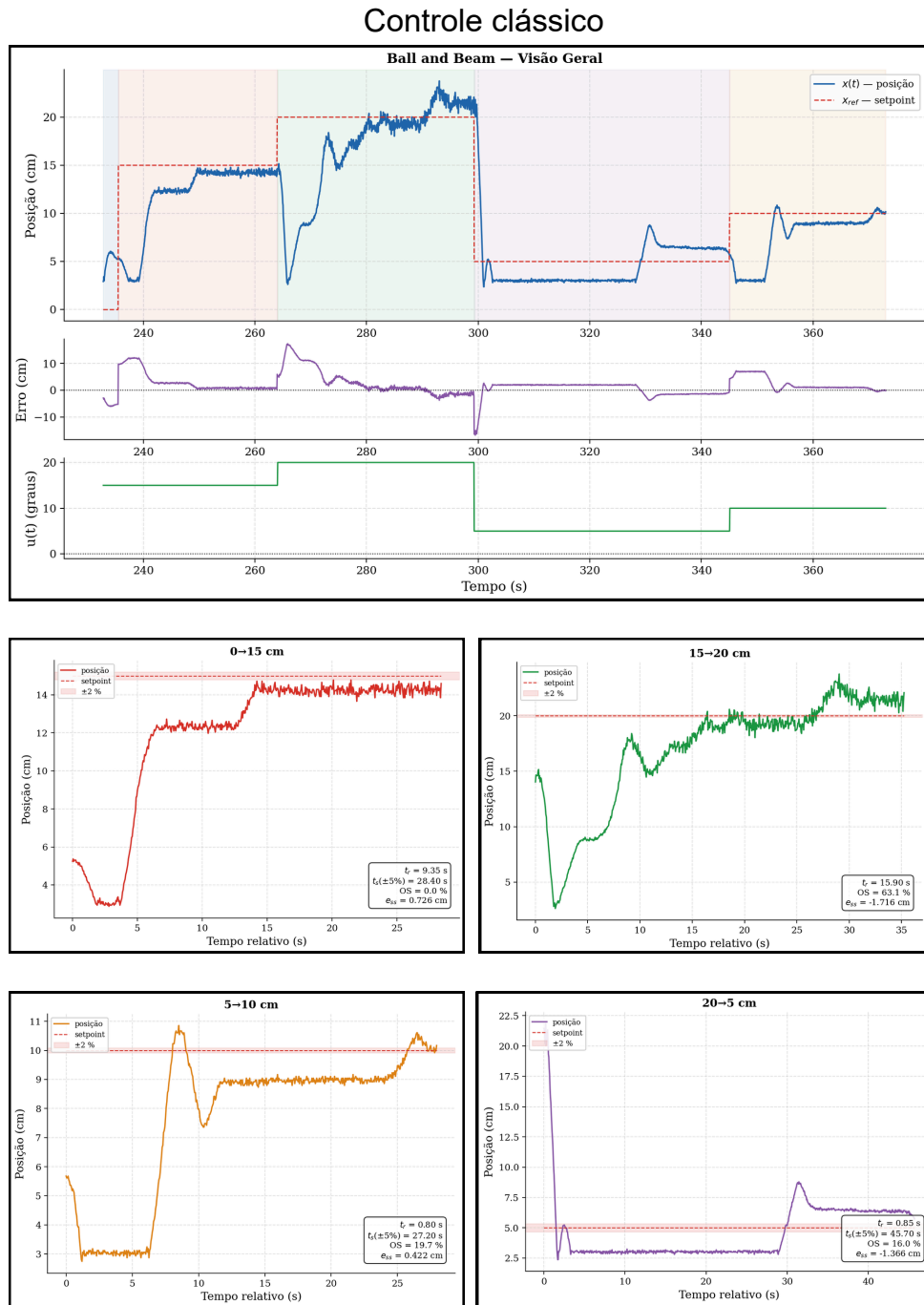


Figure 2: Painel de resultados do controlador clássico.

As métricas quantitativas utilizadas para a comparação foram: tempo de resposta (t_r), tempo de acomodação (t_s), overshoot (OS) e erro em regime permanente (e_{ss}). O controle por rede

Controle por rede neural

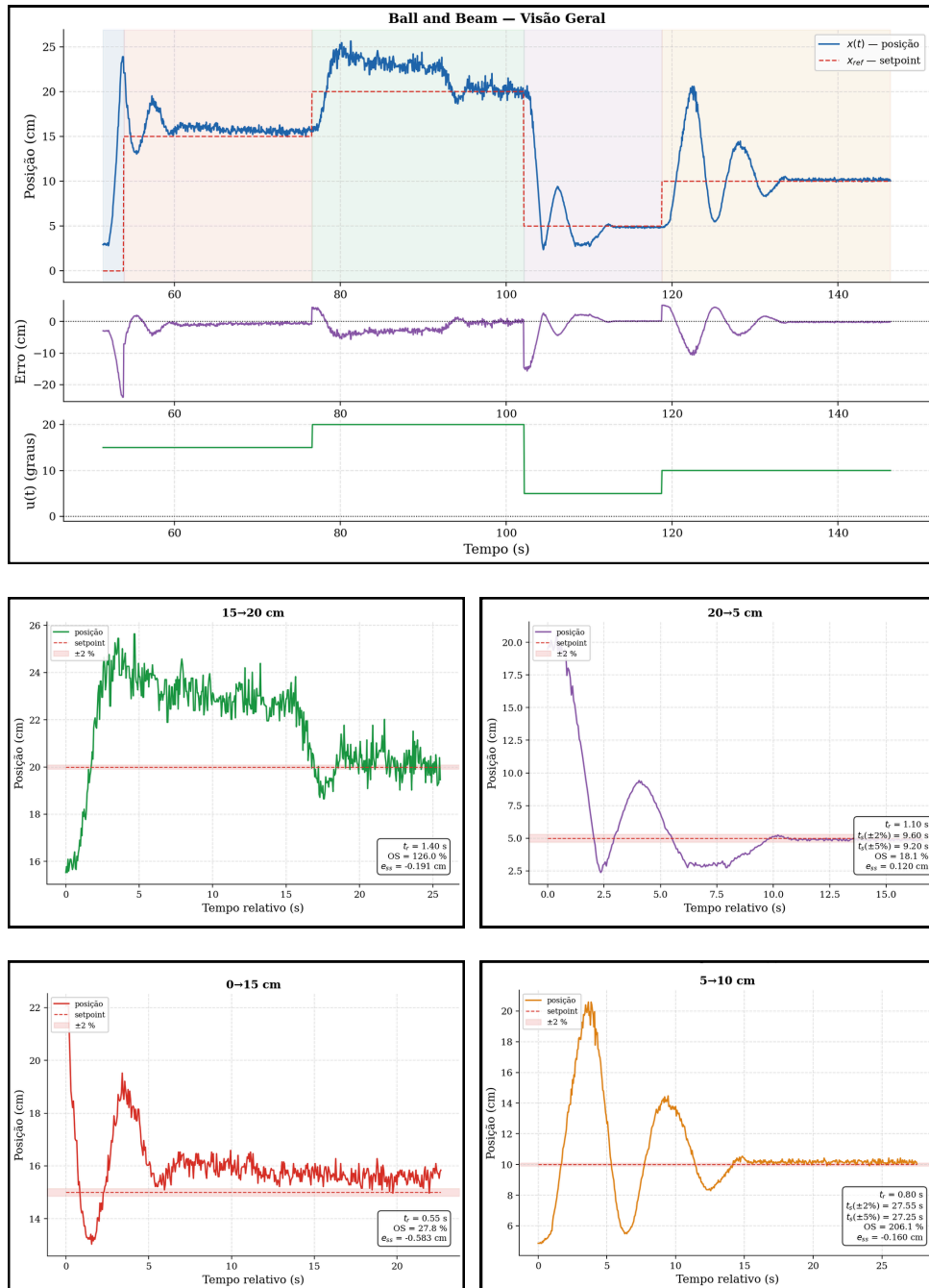


Figure 3: Painel de resultados do controlador por rede neural.

neural apresentou menores tempos de subida e de acomodação, caracterizando uma resposta mais agressiva em relação ao controlador clássico — a rede leva a bola à posição de referência com maior velocidade. O *overshoot* foi menor para o controlador clássico, refletindo seu caráter mais conservador e sua prioridade à estabilidade; vale notar, contudo, que essa métrica pode ter sido superestimada para o controle por rede neural devido à instabilidade observada nas posições mais distantes do sensor. Por fim, o erro em regime permanente foi menor para o controle por rede neural, indicando que, mesmo nas condições de maior instabilidade, a bola convergiu em

média para a posição desejada.

Não é possível determinar, de forma absoluta, qual abordagem é superior, uma vez que a escolha depende dos requisitos do sistema. No contexto deste trabalho, cujo objetivo principal era manter a bola na posição de entanto, seu comportamento mais agressivo — nem sempre desejável em aplicações práticas — e o *overshoot* mais elevado são aspectos que merecem atenção. Em particular, sistemas com restrições rígidas sobre a amplitude da ação de controle ou sobre a ultrapassagem máxima admissível exigiriam um ajuste mais cuidadoso dos pesos w_{state} e w_{effort} na função de perda, ou a incorporação de restrições explícitas durante o treinamento.

4 Conclusão

O presente trabalho demonstrou a implementação de uma rede neural capaz de aprender uma política de controle a partir exclusivamente da equação física que rege o sistema *Ball and Beam*, sem necessidade de dados experimentais rotulados. A abordagem adotada — treinamento por simulação diferenciável — contornou a principal dificuldade identificada: a impossibilidade de derivar numericamente o sinal de posição, ruidoso, para obter a aceleração necessária à formulação clássica de uma PINN. Ainda que a arquitetura resultante não corresponda estritamente a uma *Physics-Informed Neural Network*, o papel central da física permanece — embutida na dinâmica do simulador que gera as trajetórias de treinamento.

A rede foi treinada inteiramente *offline*, sem dados rotulados, e sua operação *online* produziu resultados quantitativos superiores aos do controlador clássico nos critérios estabelecidos: menor tempo de resposta, menor tempo de acomodação e menor erro em regime permanente. Ressalva-se, no entanto, que a comparação entre abordagens de controle é sempre dependente do contexto e dos requisitos da aplicação — o comportamento mais agressivo da rede, por exemplo, pode ser indesejável em sistemas com restrições rígidas sobre a ação de controle.

O uso de inteligência artificial no contexto de engenharia de sistemas de controle é um tema em franca expansão. O presente trabalho insere-se nessa perspectiva, contribuindo com uma implementação concreta de controle baseado em aprendizado de máquina em hardware embarcado de baixo custo, e apontando caminhos para desenvolvimentos futuros, como a incorporação de modelos não lineares no simulador e o uso de dados reais para reduzir o gap entre simulação e planta física.

5 Sobre o uso de Inteligência Artificial

O uso de Inteligência Artificial foi aplicado nos seguintes momentos do desenvolvimento desse trabalho:

- Auxílio na adaptação do problema de controle clássico para uma controle baseado em rede neural;

- Implementação dos seguintes *scripts*:
 1. `export_model.py` - responsável por realizar a conversão do modelo para C (.h);
 2. `generate.py` - responsável por ler e gerar *plots* de monitoramento em tempo real, além dos arquivos `csv`;
 3. `analyze_ball_beam.py` - responsável por gerar os painéis a partir dos dados salvos em `csv`
- Melhoria dos códigos referentes a *plotagem* dos dados;
- Correções de erros;
- Ajustes para melhorar a estrutura do código, com o uso de estruturas mais eficientes computacionalmente.

6 Apêndice

Dedução da equação de movimento pelo formalismo Lagrangiano

Esta seção apresenta a dedução completa da equação diferencial que governa o sistema *Ball and Beam* a partir da mecânica Lagrangiana, culminando na versão linearizada utilizada no projeto do controlador.

Coordenadas generalizadas e geometria do sistema

Adotam-se duas coordenadas generalizadas independentes:

- x — posição do centro da bola ao longo da viga, medida a partir do pivô;
- θ — ângulo de inclinação da viga em relação à horizontal.

A posição inercial do centro da bola é:

$$\mathbf{r}_b = \begin{pmatrix} x \cos \theta \\ x \sin \theta \end{pmatrix}, \quad (5)$$

de modo que a velocidade do centro de massa da bola vale:

$$v_b^2 = \dot{x}^2 + x^2 \dot{\theta}^2. \quad (6)$$

Energia cinética

A energia cinética total do sistema é composta por três parcelas: translação da bola, rotação da bola em torno do próprio eixo e rotação da viga em torno do pivô.

Translação da bola.

$$T_{\text{trans}} = \frac{1}{2} m v_b^2 = \frac{1}{2} m (\dot{x}^2 + x^2 \dot{\theta}^2). \quad (7)$$

Rotação da bola. Para uma esfera sólida, o momento de inércia em relação ao centro de massa é $I_b = \frac{2}{5} m r^2$. A condição de rolamento sem deslizamento impõe $\dot{\phi} = \dot{x}/r$, logo:

$$T_{\text{rot},b} = \frac{1}{2} I_b \dot{\phi}^2 = \frac{1}{2} \cdot \frac{2}{5} m r^2 \cdot \frac{\dot{x}^2}{r^2} = \frac{1}{5} m \dot{x}^2. \quad (8)$$

Rotação da viga. Sendo J o momento de inércia da viga em relação ao pivô:

$$T_{\text{viga}} = \frac{1}{2} J \dot{\theta}^2. \quad (9)$$

Energia cinética total.

$$T = \frac{7}{10} m \dot{x}^2 + \frac{1}{2} (m x^2 + J) \dot{\theta}^2. \quad (10)$$

Energia potencial

Tomando o nível do pivô como referência gravitacional:

$$V = m g x \sin \theta. \quad (11)$$

Lagrangiano

O Lagrangiano do sistema é $\mathcal{L} = T - V$:

$$\mathcal{L} = \frac{7}{10} m \dot{x}^2 + \frac{1}{2} (m x^2 + J) \dot{\theta}^2 - m g x \sin \theta. \quad (12)$$

Equações de Euler-Lagrange

As equações de movimento são obtidas pela aplicação do operador

$$\frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{q}_i} - \frac{\partial \mathcal{L}}{\partial q_i} = Q_i, \quad (13)$$

onde Q_i é a força/torque generalizado associado à coordenada q_i .

Coordenada x ($Q_x = 0$, sem forçamento direto na direção da viga).

$$\frac{\partial \mathcal{L}}{\partial \dot{x}} = \frac{7}{5} m \dot{x}, \quad \frac{d}{dt} \left(\frac{7}{5} m \dot{x} \right) = \frac{7}{5} m \ddot{x}, \quad (14)$$

$$\frac{\partial \mathcal{L}}{\partial x} = m x \dot{\theta}^2 - m g \sin \theta. \quad (15)$$

A equação resultante é:

$$\boxed{\frac{7}{5}m\ddot{x} = mx\dot{\theta}^2 - mg \sin \theta.} \quad (I)$$

Coordenada θ (torque de controle τ).

$$\frac{\partial \mathcal{L}}{\partial \dot{\theta}} = (mx^2 + J) \dot{\theta}, \quad (16)$$

$$\frac{d}{dt} \left[(mx^2 + J) \dot{\theta} \right] = 2mx\dot{x}\dot{\theta} + (mx^2 + J) \ddot{\theta}, \quad (17)$$

$$\frac{\partial \mathcal{L}}{\partial \theta} = -mgx \cos \theta. \quad (18)$$

A equação resultante é:

$$(mx^2 + J) \ddot{\theta} + 2mx\dot{x}\dot{\theta} + mgx \cos \theta = \tau. \quad (II)$$

Linearização em torno do ponto de equilíbrio

Considera-se operação próxima ao equilíbrio $\theta = 0$, com os ângulos assumidos suficientemente pequenos para que as seguintes aproximações sejam válidas:

$$\sin \theta \approx \theta, \quad \cos \theta \approx 1, \quad x\dot{\theta}^2 \approx 0, \quad \ddot{\theta} \approx 0. \quad (19)$$

O ângulo θ é tratado como variável de entrada (imposta pelo servomotor), de modo que a dinâmica de interesse reduz-se à equação (I). Aplicando as aproximações acima:

$$\frac{7}{5}m\ddot{x} = -mg\theta \implies \ddot{x} = -\frac{5g}{7} \theta. \quad (20)$$

Adotando $g \approx 9,8 \text{ m/s}^2$ e a convenção de sinal em que $\theta > 0$ produz deslocamento positivo de x , e aproximando o coeficiente para $5/9$ (valor frequentemente empregado na literatura de controle deste sistema), obtém-se a equação utilizada no projeto:

$$\ddot{x} = \frac{5}{9} \theta. \quad (1)$$

Referências

OCHOA, Eduardo. **Ball and Beam 2**. [S. l.: s. n.], 2022. Modelo CAD disponível na plataforma GrabCAD. Disponível em: <https://grabcad.com/library/ball-and-beam-2>. Acesso em: 6 jun. 2026.

OGATA, Katsuhiko; SOUZA, Heloisa Coimbra de; TANNURI, Eduardo Aoun. **Engenharia de controle moderno**. 5. ed. [S. l.]: Pearson Prentice Hall, 2011.

RAISSI, Maziar; PERDIKARIS, Paris; KARNIADAKIS, George E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. **Journal of Computational physics**, Elsevier, v. 378, p. 686–707, 2019.